

SOFTWARE ENGINEERING FOR ML · COMPANION READER

CQRS, RAG, Monolith & Microservices

Four architectural patterns for building ML systems that scale

Session 5 · Read alongside the lecture slides · Every slide example worked out in full

How to use this companion. The slides give you the formulas and diagrams; this reader gives you the *why*. Every concept is expanded into a full, plain-English explanation. Every slide example is worked out step by step. Five colour-coded boxes guide you through each concept. The **blue** box states the core idea. The **amber** box ties it to real life. The **green** box solves a problem step by step. The **red** box flags the common mistake. The **purple** box gives the one line to remember.

1 What this session covers

This session teaches four architectural patterns. They show up often in ML systems and in software engineering more broadly. We go in a natural order. First: the idea of separating reads from writes (CQRS). Then: a real system built on that idea (RAG). Then: two ways to package an entire application (monolith and microservices).

- **CQRS** — Command Query Responsibility Segregation. Keep writes and reads on separate paths.
- **RAG** — Retrieval-Augmented Generation. Ground an LLM in fresh documents at query time.
- **Monolith** — all code ships as one unit.
- **Microservices** — each job gets its own independent service.

2 CQRS: keeping reads and writes separate

Imagine a busy library. People come in to either borrow a book (they change the stock) or check the catalogue (they just read it). If both groups use the same counter, they slow each other down. **CQRS** (Command Query Responsibility Segregation) says: give them separate counters.

The idea comes from **Bertrand Meyer**'s CQS (Command–Query Separation) principle, introduced in the Eiffel programming language. **Greg Young** later extended it into CQRS as an architectural pattern.

The rule, quoted from *Cloud Native Patterns* by Cornelia Davis, is:

“Every method should either be a command that performs an action, or a query that returns data to the caller, but not both.”

A **command** changes the world — it creates, updates, or deletes something. A **query** reads the world — it returns data without changing anything.

The intuition

CQRS is the single-responsibility principle applied to operations. A command is a write; a query is a read. They never do both at the same time. Keep them on separate paths, with separate stores if needed.

★ Everyday picture

Think of an ATM and a bank's website. The ATM handles commands: withdraw cash, deposit cheques. The website handles queries: show your balance, list recent transactions. Both hit the same account, but they use different interfaces. The website does not lock your account just to show your balance. That is CQRS in daily life.

2.1 CQRS and HTTP verbs

In a web API, HTTP methods map directly to commands and queries. GET is the only safe method: call it a hundred times and nothing changes. POST, PUT, and DELETE all change state.

HTTP method	Operation	CQRS side
GET	Retrieve a resource	Query (read; no state change)
POST	Create a new resource	Command (write)
PUT	Update or replace a resource	Command (write)
DELETE	Remove a resource	Command (write)

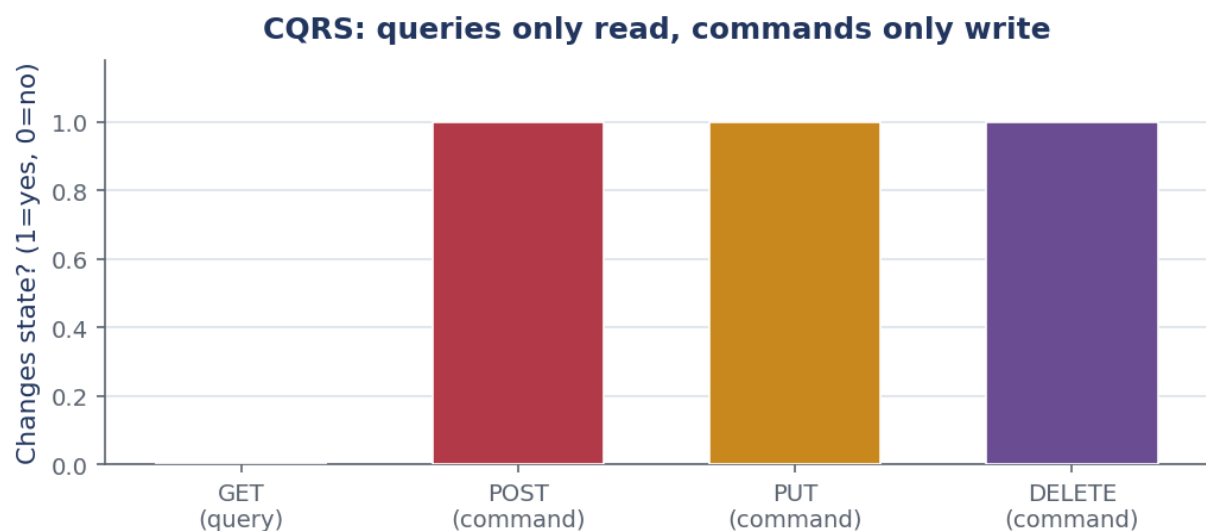


Figure 1: HTTP verbs mapped to CQRS sides. Only GET is a query; POST, PUT, and DELETE are commands that change state.

2.2 CQRS and the data store

In a standard design, one database handles every read and every write. This creates **data contention**: two operations fighting over the same rows.

Databases protect shared rows with **locks**:

- A **shared lock** (S-lock) lets many readers in at once, but blocks any writer.
- An **exclusive lock** (X-lock) blocks everyone else while one writer works.

Under heavy load, a slow write makes every reader wait. CQRS solves this by using **two separate stores**. One store is optimised for writes; the other is optimised for reads. The write store is the source of truth. Changes flow from the write store to the read store in the background.

CQRS: write and read stores are kept separate

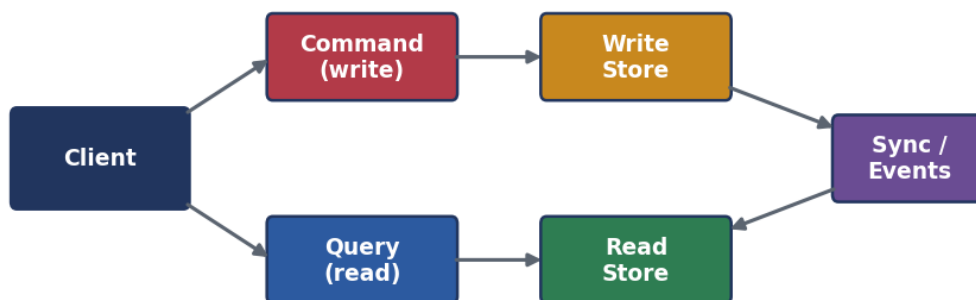


Figure 2: CQRS architecture. Commands go to the write store; queries go to the read store. A sync or event layer keeps the read store up to date.

2.3 Eventual consistency

When you have two stores, they are not in sync at every instant. There is a short window when the write store knows about a new fact but the read store does not yet.

Eventual consistency is the guarantee that this gap closes. All parts of the system will show the same data given enough time, even if they briefly disagree.

The intuition

Eventual consistency is like posting a letter. The moment you drop it in the postbox, the sender knows the letter is gone. The receiver has not got it yet. But they will — eventually, both sides agree.

Everyday picture

You post a photo on social media. Friends in the same city see it in seconds. A friend in another country may see it a minute later. For a short window, different people see different states. Eventually all feeds agree. That is eventual consistency.

Worked example: ChatGPT 6.0 rolls out on 1 June 2026 at 12:00 PM

OpenAI deploys a new model version. We trace what different parts of the world see.

Step 1. Write-side update. The master model registry records version = 6.0 at 12:00:00. This is the command-side update.

- Step 2. US-East inference servers.** The nearest cluster syncs the update. New York users see model 6.0 by 12:00:05.
- Step 3. EU servers.** The Frankfurt region pulls the update. European users get model 6.0 at 12:01:12.
- Step 4. Asia-Pacific.** Singapore and Tokyo sync. At 12:02:00 some users still see model 5.x.
- Step 5. Edge caches.** CDN caches expire and mobile-app cached responses clear. All users worldwide see model 6.0 by 12:03:30.
- Step 6. Conclusion.** From 12:00 to 12:03:30 the system was in an eventually consistent state. No single moment had all regions in sync at once. After about 3.5 minutes, every endpoint agreed. **This is a deliberate trade-off, not a bug.**

Eventual consistency: each region updates at a different time

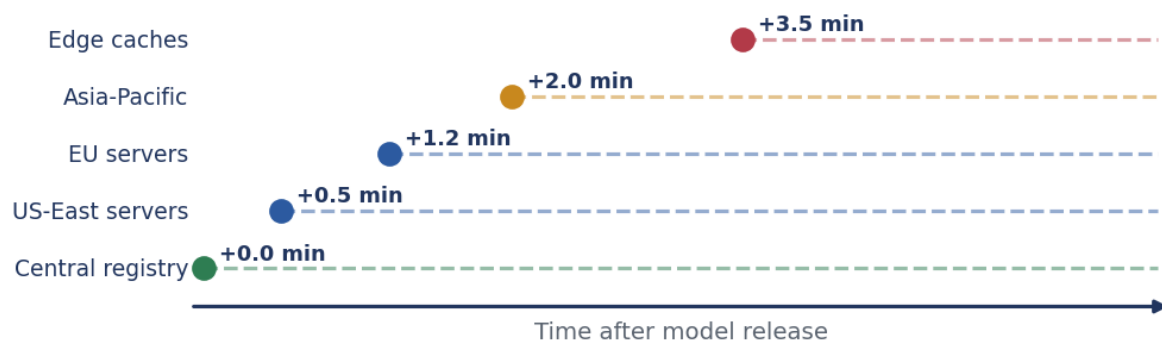


Figure 3: Eventual consistency timeline. Each region updates at a different time after the model release. The gap closes; all regions eventually agree.

2.4 CQRS in ML systems

In an ML system, the two CQRS sides map to training and serving:

Side	ML activity	Tools
Command (write)	Update data, models, features	Apache Kafka, Airflow, Prefect
Query (read)	Serve predictions, metrics, dashboards	REST APIs, dashboards, batch scoring

The two sides talk through **event streams** (Apache Kafka, Amazon Kinesis) or **message brokers** (RabbitMQ, ActiveMQ). A training job finishes and pushes a “new model ready” event. The serving layer picks up the event and hot-swaps the model. They never share a lock.

Where this shows up in ML. Any production ML system with a separate training pipeline and serving API uses this split. Feature stores (Feast, Tecton) follow the same CQRS idea: writes come from offline feature computation; reads come from the online serving layer. Keeping them apart means a slow training run never blocks a live prediction.

✗ Watch out

Do not confuse “eventual consistency” with “data loss.” The data is never lost; it just has not reached every node yet. Also, not every system needs separate stores. A simple prototype can use one database. CQRS adds complexity; use it only when the write/read load or latency mismatch justifies it.

✓ Key takeaway

CQRS puts commands (writes) on one path and queries (reads) on another. The write store is the source of truth. The read store is optimised for fast serving. Eventual consistency bridges the two.

3 RAG: grounding an LLM in fresh documents

A large language model (LLM) knows a lot, but its knowledge stops at a training cutoff date. Ask it about a document your company wrote last week and it cannot answer. It has never seen that document.

RAG (Retrieval-Augmented Generation) solves this without retraining. At query time, you fetch the relevant document and hand it to the model alongside the question. The model reads the fetched context and answers from it. This also reduces **hallucinations**: the model is less likely to make things up when real text is in front of it.

🧠 The intuition

RAG is like giving the model an open-book exam instead of a closed-book one. The model looks things up in a curated folder of documents before it answers. It does not need to memorise everything; it just needs to read and reason.

★ Everyday picture

You need to answer a customer’s question about a contract signed three days ago. You do not memorise every contract. You search your filing system, pull the right contract, read the relevant clause, and then answer. RAG does exactly that: search, retrieve, then generate.

3.1 RAG as two CQRS pipelines

A RAG system has two separate pipelines. They map directly onto CQRS: the write side ingests documents; the read side answers queries.

Ingestion pipeline (command / write side):

1. **Documents** — your raw source files (PDFs, web pages, notes).
2. **Extraction** — pull the plain text out of each file.
3. **Chunking** — split long texts into short, overlapping pieces.
4. **Embedding generation** — turn each chunk into a dense vector that encodes its meaning.
5. **Indexing** — store those vectors in a searchable index.

6. **Vector store** — the database that holds all vectors and their source text.

Query pipeline (query / read side):

1. **User query** — the question in plain text.
2. **Query embedding** — turn the query into a vector with the same model.
3. **Semantic search** — find the vectors in the store closest to the query.
4. **Context retrieval** — fetch the original text chunks for those close vectors.
5. **LLM prompt construction** — build a prompt: “Here is some context. Answer this question: ...”
6. **Answer generation** — send the prompt to the LLM and return its response.

Both pipelines also map to the **Pipe-and-Filter** pattern. Each stage is a filter: it takes input, transforms it, and passes the result on.

RAG maps onto two CQRS pipelines

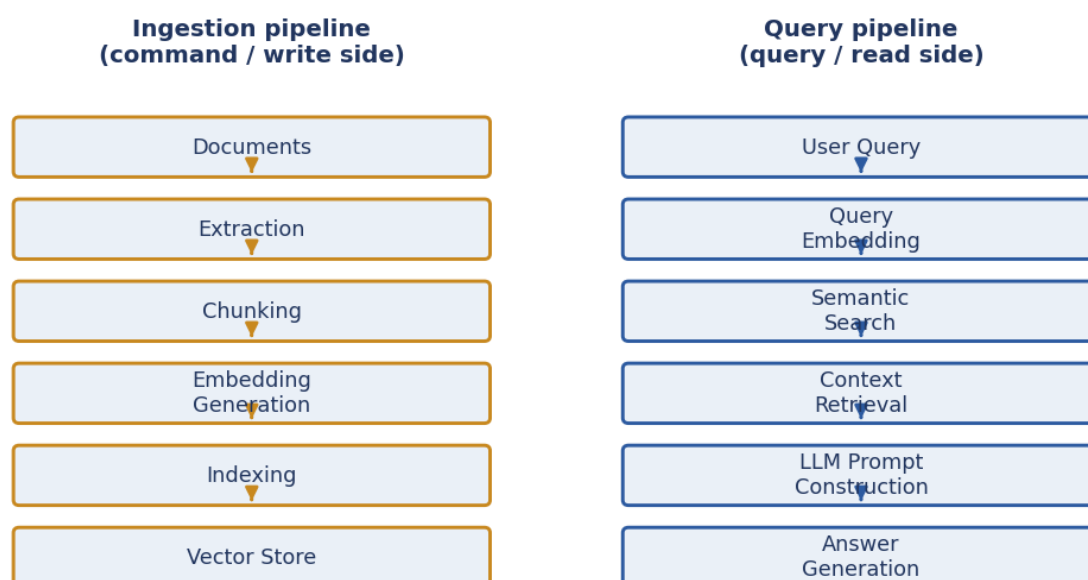


Figure 4: RAG as two CQRS pipelines. Left: ingestion (write side) stores documents as vectors. Right: query (read side) retrieves relevant chunks and generates an answer.

3.2 What is an embedding?

An **embedding** is a list of numbers (a *vector*) that captures the meaning of a piece of text. Two pieces of text with similar meaning have vectors that are close together in space. Two pieces with very different meaning have vectors that are far apart.

We measure closeness with **cosine similarity**. For two vectors **u** and **v**:

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} \quad (1)$$

Here $\mathbf{u} \cdot \mathbf{v}$ is the dot product (sum of element-wise products), and $\|\mathbf{u}\|$ is the length of the vector. A result near 1 means the two texts are similar; near 0 means they are unrelated.

Worked example: cosine similarity between two short sentences

Say we embed two sentences into 3-dimensional vectors (real embeddings have 768+ dimensions, but 3 is enough to see the math).

Sentence A: “CQRS separates reads from writes.”

Sentence B: “Commands and queries use different stores.”

Sentence C: “Python is a programming language.”

After embedding (using illustrative numbers):

$$\mathbf{a} = [0.6, 0.5, 0.2], \quad \mathbf{b} = [0.7, 0.4, 0.3], \quad \mathbf{c} = [-0.1, 0.2, 0.9].$$

Step 1. Dot product of A and B.

$$\mathbf{a} \cdot \mathbf{b} = (0.6)(0.7) + (0.5)(0.4) + (0.2)(0.3) = 0.42 + 0.20 + 0.06 = 0.68.$$

Step 2. Lengths of A and B.

$$\|\mathbf{a}\| = \sqrt{0.6^2 + 0.5^2 + 0.2^2} = \sqrt{0.36 + 0.25 + 0.04} = \sqrt{0.65} \approx 0.806.$$

$$\|\mathbf{b}\| = \sqrt{0.49 + 0.16 + 0.09} = \sqrt{0.74} \approx 0.860.$$

Step 3. Cosine similarity of A and B.

$$\cos(\theta_{AB}) = \frac{0.68}{0.806 \times 0.860} = \frac{0.68}{0.693} \approx \mathbf{0.98}.$$

Very close to 1: sentences A and B are about the same topic.

Step 4. Cosine similarity of A and C.

$$\mathbf{a} \cdot \mathbf{c} = (0.6)(-0.1) + (0.5)(0.2) + (0.2)(0.9) = -0.06 + 0.10 + 0.18 = 0.22.$$

$$\|\mathbf{c}\| = \sqrt{0.01 + 0.04 + 0.81} = \sqrt{0.86} \approx 0.927.$$

$$\cos(\theta_{AC}) = \frac{0.22}{0.806 \times 0.927} = \frac{0.22}{0.747} \approx \mathbf{0.29}.$$

Much lower: sentences A and C are about different topics.

Step 5. RAG uses this. When a user asks “how does CQRS work?”, the query vector is close to **a** and **b**. The vector store returns chunks A and B as context. Chunk C (about Python) is ignored. **The LLM then answers from the retrieved context.**

Common embedding models used in RAG:

Model	Key property
GloVe	Static word vectors; no context. The word “bank” always maps to the same vector.
BERT	Contextual; the same word gets different vectors in different sentences.
Sentence-BERT	Optimised for sentence and paragraph embeddings; strong for retrieval.
DPR (Dense Passage Retriever)	Trained for question-answer retrieval tasks.
OpenAI Embeddings	High-quality general embeddings; called via API.

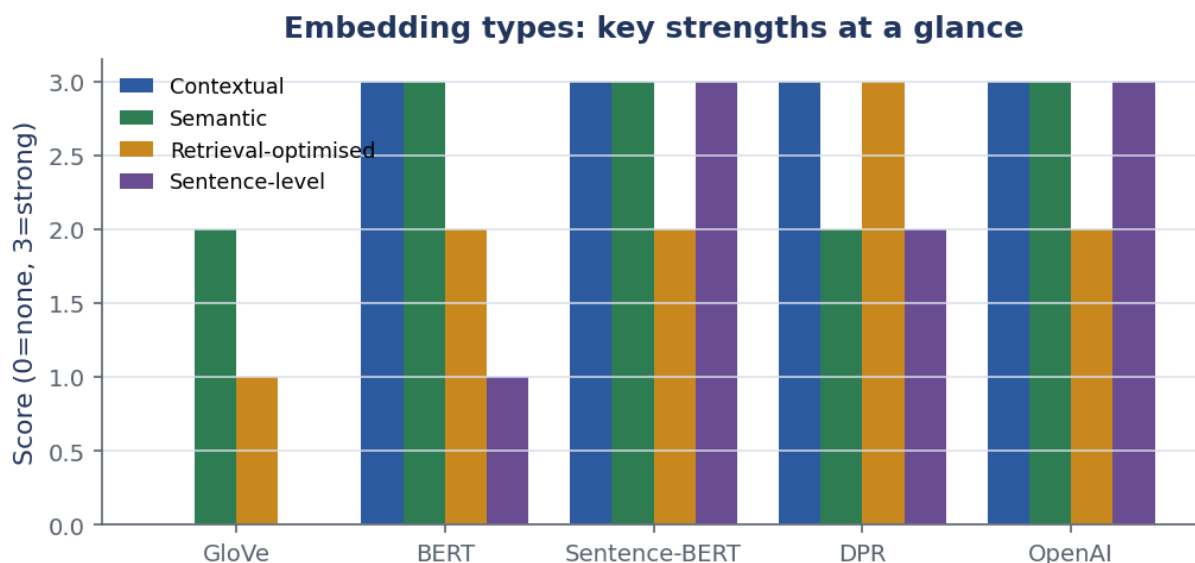


Figure 5: Embedding types compared on four strengths. Sentence-BERT and DPR score highest on retrieval and sentence-level tasks.

3.3 Vector stores

A **vector store** is a database built for one job: find the vectors nearest to a query vector, fast. It handles millions of high-dimensional vectors (often 768 or 1536 numbers each) and returns the top-K closest ones in milliseconds.

Store	Best for
Pinecone	Managed cloud service; scales to billions of vectors with no ops overhead.
FAISS	Open-source; fastest on-machine search; no built-in persistence layer.
ChromaDB	Local-first; wraps SQLite and HNSW; great for dev and small RAG projects.
ElasticSearch + KNN	Adds vector search to a standard text search engine.

3.4 Inside ChromaDB

ChromaDB stores each chunk in two places:

- **SQLite** holds the text, IDs, and metadata (source file, page, chunk number).
- **HNSW index** (*Hierarchical Navigable Small World*) holds the embedding vectors and supports fast approximate nearest-neighbour search.

An HNSW index is a layered graph. The top layer has a few widely spaced nodes. Each lower layer is denser. To find the nearest vector to a query, start at the top. Hop to the closest node, then drop to a finer layer and repeat. This gives approximate nearest neighbours in $O(\log N)$ time, where N is the number of stored vectors.

Worked example: one stored document chunk in ChromaDB

We chunk a PDF and store the first piece.

Step 1. Input text. “The CQRS pattern separates reads from writes.” This is page 1, chunk 1 of a PDF called `session5.pdf`.

Step 2. Generate an embedding. We call Sentence-BERT. It returns a vector of 768 numbers. We write it as $\mathbf{e} = [0.23, -0.11, \dots]$ (768 values in total).

Step 3. Build the ChromaDB document record.

```
{
  "ID": "chunk_1",
  "Document": "The CQRS pattern separates reads from writes.",
  "Embedding": [0.23, -0.11, ...],
  "Metadata": {
    "source": "session5.pdf",
    "page": 1,
    "chunk": 1
  }
}
```

Step 4. SQLite write. The ID, Document text, and Metadata go into a SQLite table row.

Step 5. HNSW insert. The Embedding vector goes into the HNSW graph. The index adds edges to the existing nearest vectors.

Step 6. Retrieval check. A later query “what does CQRS do?” gets embedded. Its vector is close to $\mathbf{e}$. The HNSW index returns `chunk_1` as a top match. ChromaDB fetches the text from SQLite and returns it as context. **The LLM reads this chunk and answers the question.**

ChromaDB stores text in SQLite and embeddings in an HNSW index



Figure 6: ChromaDB pipeline. Text moves through chunking and embedding before being stored in SQLite (text) and the HNSW index (vectors). A query traverses the HNSW index to find the nearest vectors, then fetches text from SQLite.

3.5 LangChain: the glue framework

LangChain is an open-source Python framework. It chains together the stages of a RAG pipeline — and other LLM applications — with minimal boilerplate. It provides connectors for document loaders, text splitters, embedding models, vector stores, and LLMs. You can wire up

a full RAG system in a few dozen lines.

Where this shows up in ML. RAG is now the standard way to add private or up-to-date knowledge to an LLM. Every enterprise chatbot that answers questions about internal documents uses some form of RAG. The CQRS split inside RAG means the ingestion pipeline can run overnight while the query pipeline serves live users. They never block each other. Vector stores also power semantic search, recommendation engines, and image retrieval.

✗ Watch out

Chunking strategy matters a lot. Chunks that are too short lose context; chunks that are too long dilute the signal. A common starting point is 500 tokens per chunk with 50-token overlap. Also, the embedding model at ingestion time must match the one at query time. Using different models gives meaningless similarity scores.

✓ Key takeaway

RAG = retrieve first, then generate. It avoids retraining and reduces hallucinations. The two RAG pipelines map onto CQRS: ingestion is the write side; query is the read side. A vector store is the bridge between them.

4 Monolithic architecture

The word **monolith** comes from the Greek for “one stone.” In software, a monolithic application packages all its functions into a single deployable unit. The UI, the business logic, and the database access layer all ship together.

🔗 The intuition

A monolith is a Swiss Army knife. Everything you need is in one tool. Opening one blade does not affect the others, but the whole knife is one object you buy, maintain, and replace together.

★ Everyday picture

Think of a small restaurant where one chef cooks everything: starters, mains, desserts. The kitchen has one entrance and one set of equipment. It is easy to manage when small. But if dessert orders surge, you cannot hire just a pastry chef. You must hire a whole new kitchen team. That is the scaling problem of a monolith.

4.1 The FTGO monolith example

The slides use **FTGO** (Food to Go), a food delivery app like Swiggy or Zomato. In a monolithic design:

- All modules (orders, payments, notifications, delivery) live in **one codebase**.
- Everything deploys as a **single WAR file** (one bundle).
- One **MySQL database** is shared by every module.
- External services (Twilio for SMS, AWS SES for email, Stripe for payments) connect via adapters in the same codebase.

 Worked example: FTGO: a single module needs more capacity

Suppose FTGO runs a Friday-night discount. Payment requests spike to 10× normal load.

Step 1. Identify the bottleneck. The payment module is slow. We want to run more copies of it.

Step 2. The monolith constraint. We cannot deploy more copies of just the payment module. The whole app is one unit. To add capacity we must deploy a full copy of the entire application.

Step 3. What that means in practice. Each full copy includes the order module, the notification module, the delivery module. All of them run at normal load and do not need scaling. We pay for the whole app to get more of one part.

Step 4. Conclusion. Monolith scaling is all-or-nothing. This wastes resources when only one part of the system is hot.

4.2 Monolith: advantages

- **Performance.** All components share memory. A call between modules is a local function call, not a network request.
- **Easy to test.** End-to-end tests run against one process. No inter-service mocking is needed.
- **Easy to deploy.** One WAR file, one command.

4.3 Monolith: limitations

- **Technology lock-in.** The whole app uses one language and one framework. Adopting something new may mean rewriting everything.
- **Coarse scaling.** You must scale the whole app, not just the busy module.
- **Hard to understand.** A new developer must learn the whole system, not just their area, because modules share code and data.
- **Single point of failure.** A bug in one module can crash the whole app.

4.4 Monolithic ML pattern

An ML monolith puts everything in one codebase. That means the HTTP API, the feature preprocessing, the model loading, and the prediction logic all ship together.

Advantages for ML prototypes:

- No network hops between components (lower latency).
- One deployment for the whole pipeline.
- Good for small projects and experiments.

Limitations for production ML:

- A traffic spike forces you to scale the entire app, including the training pipeline that is not serving requests.
- A model update requires redeploying the whole codebase.

Monolith packs everything together; microservices split it apart

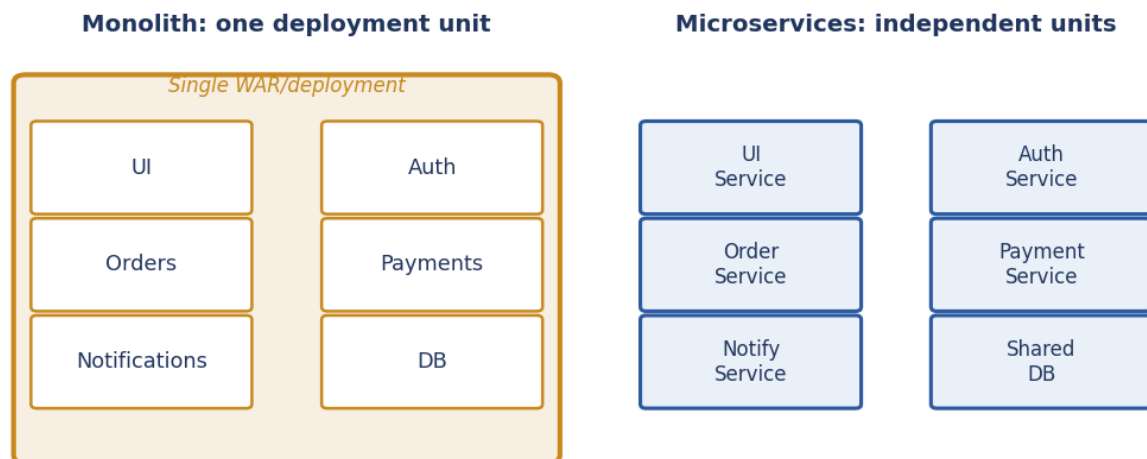


Figure 7: Monolith vs microservices at a glance. Left: all components are packed into one deployment unit. Right: each component is its own independent island that can scale and deploy separately.

- One failing model crashes the entire service.

Where this shows up in ML. Most ML projects start as monoliths. A Jupyter notebook or a FastAPI app that loads the model and serves predictions is a monolith. The switch to microservices happens when one part of the system needs to scale independently of the rest, typically the model server.

⚠ Watch out

“Monolith” is not a bad word. Many successful ML systems run as monoliths for years. The problems appear at scale. Start simple and refactor only when you hit the limits. Do not add microservices complexity before you need it.

✓ Key takeaway

A monolith ships everything as one unit. It is simple to build and deploy. Its main weakness is scaling: you cannot scale one part without scaling all of it.

5 Microservices architecture

A **microservices** architecture breaks an application into small, independent services. Each service does one job, runs in its own process, and communicates with others over a network API.

Google’s definition:

“A microservices architecture is a type of application architecture where the application is developed as a collection of services. It provides the framework to develop, deploy, and maintain microservices architecture diagrams and services independently.”

The design philosophy comes from **Robert C. Martin’s Single Responsibility Principle**:

“Gather together the things that change for the same reasons. Separate those things

that change for different reasons.”

In plain words: if two pieces of code always change together, keep them in one service. If they change for different reasons, put them in different services.

The intuition

A microservices app is like a food court. Each stall does one thing: one sells pizza, one sells sushi, one sells coffee. Each stall can open, close, or hire more staff independently. If the pizza stall is busy, only the pizza stall gets a longer counter. The sushi stall does not change at all.

★ Everyday picture

Instagram is built with Python, JavaScript, React Native, Django, PostgreSQL, Objective-C, Nginx, and Redis. Different teams use different tools for different jobs. The notification team can deploy a fix without touching the photo-upload team. Each service owns its own database and its own deployment pipeline. That is microservices in practice.

5.1 Why use microservices?

- **Faster deployment cycles.** Large organisations run 500 or more micro-deployments per day. Each team ships on its own schedule.
- **Polyglot stacks.** Different services can use different languages and frameworks. A data-processing service might use Python; a real-time API might use Go.
- **Independent scaling.** Only the service under load needs more instances.
- **Fault isolation.** A crash in the payment service does not take down the order service.

5.2 FTGO as microservices

The same FTGO food delivery app, split into microservices:

- **Order Service** — creates and tracks orders.
- **Payment Service** — charges the customer.
- **Notification Service** — sends SMS and email updates.
- **Delivery Service** — manages courier assignment.
- **API Gateway** — the single entry point; routes requests to the right service.

Services talk to each other using:

- **REST APIs** over HTTP — simple and widely supported.
- **gRPC** — a faster binary protocol, good for internal service-to-service calls.
- **GraphQL** — flexible query language, useful when clients need custom shapes.
- **Async messaging** (Kafka, RabbitMQ) — for events that do not need an instant reply.

 Worked example: FTGO microservices: the Friday-night payment spike

The same scenario, now with microservices.

- Step 1. Identify the bottleneck.** Payment Service is under 10× normal load.
- Step 2. Scale only the hot service.** We start 5 more instances of `payment-service` in Kubernetes. The `order-service` and `notification-service` stay at 2 instances each.
- Step 3. Cost comparison.** We pay only for 5 extra payment-service instances. All other services are untouched. In the monolith, we would have paid for 5 full copies of everything.
- Step 4. Deploy an update independently.** The payments team finds a bug at 2 AM. They ship a fix to `payment-service` alone. Zero other services restart. Zero downtime for orders or notifications.
- Step 5. Conclusion. Fine-grained scaling and independent deployment** are the two main wins. Both are impossible in a monolith.

Monolith packs everything together; microservices split it apart

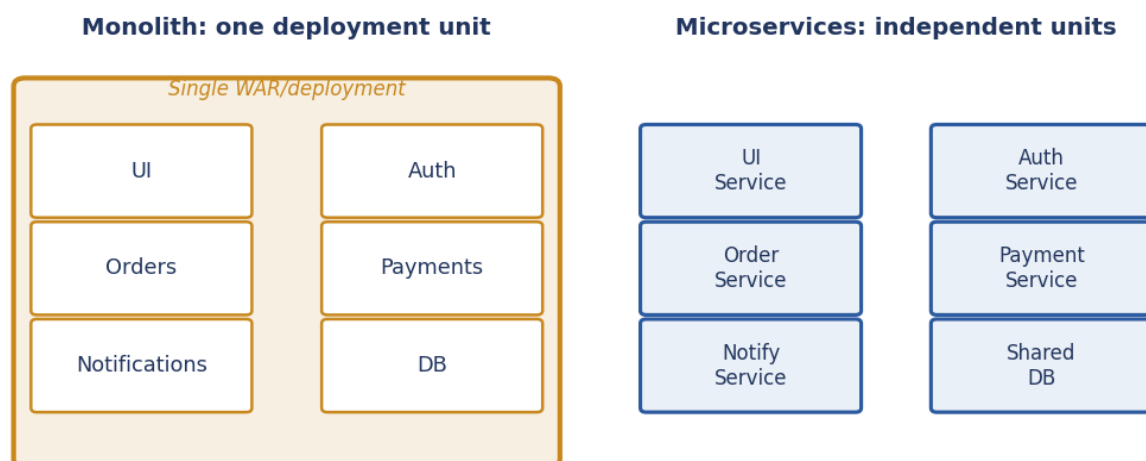


Figure 8: Monolith vs microservices. In the monolith, all components sit inside one deployment unit. In microservices, each component is its own independent island.

5.3 Microservices ML pattern

In an ML system, microservices give each ML function its own service:

Service	Single responsibility
Preprocessing service	Clean and transform raw input features.
Model inference service	Load the model and run predictions.
Feature store service	Serve pre-computed features at low latency.
Logging / observability	Record predictions for monitoring and retraining.
API gateway	Route requests; handle auth and rate limiting.

The demo in the slides uses three services on one machine:

- **Gateway** at `localhost:8000` — routes every incoming request.

- **Model Service** at `localhost:8001` — loads the ML model and runs inference.
- **Logging Service** at `localhost:8002` — records every prediction for monitoring and later retraining.

 Worked example: a prediction request through the 3-service ML system

The client sends `POST /predict` with input features.

- Step 1. Gateway (:8000) receives the request.** It checks authentication and routes the request to Model Service. It does not touch the model; routing is its only job.
- Step 2. Model Service (:8001) runs inference.** It calls `model.predict(features)` on the loaded model. It returns a JSON response: `{"prediction": 0.87}`.
- Step 3. Gateway forwards the response to the client.** Simultaneously, it sends a copy of the request and result to Logging Service. The client does not wait for logging to finish.
- Step 4. Logging Service (:8002) records the event.** It writes the timestamp, input features, and prediction to a log store. This runs in the background.
- Step 5. Result.** The client got its answer from Model Service. Logging Service captured the event for later analysis. **Each service did exactly one job.**

Three-service ML system: each service has one job

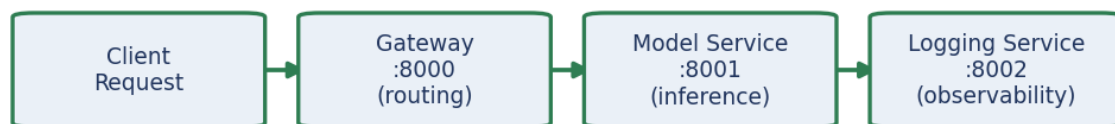


Figure 9: Three-service ML system. The gateway routes requests; the model service runs predictions; the logging service records outcomes. Each service has one job and its own deployment lifecycle.

Where this shows up in ML. Major ML platforms (SageMaker, Vertex AI, MLflow) use a microservices design under the hood. A model serving endpoint is its own service. The feature store is its own service. The experiment tracker is its own service. Teams working on research (model service) and teams working on data (feature service) deploy on their own schedules without blocking each other.

 Watch out

Microservices bring real costs. Each service adds network latency, a new deployment pipeline, and more monitoring to set up. Debugging a request that touches five services is harder than debugging one process. Start with a monolith. Find where the seams are. Extract services one at a time when the need is real.

✓ Key takeaway

Microservices apply SRP at the system level. Each service does one job, deploys independently, and scales independently. The trade-off is added network complexity. Start monolith; evolve to microservices when you need to.

Glossary & symbols

Key terms.

CQRS	Command Query Responsibility Segregation: every operation is either a command (write) or a query (read), never both.
Command	An operation that changes state (POST, PUT, DELETE).
Query	An operation that reads state without changing it (GET).
Eventual consistency	The guarantee that all nodes in a distributed system will agree on the same data after some time passes.
S-lock	Shared lock: many readers can hold it at once.
X-lock	Exclusive lock: only one writer holds it; all others wait.
RAG	Retrieval-Augmented Generation: fetch relevant documents, inject them into the prompt, then generate an answer.
Embedding	A dense vector of numbers that encodes the meaning of a text chunk.
Vector store	A database built for fast nearest-neighbour search over embeddings.
ChromaDB	A local-first vector store using SQLite for text and HNSW for vectors.
HNSW	Hierarchical Navigable Small World: a graph-based index for fast approximate nearest-neighbour search.
LangChain	An open-source framework for building LLM-powered applications.
Monolith	All application components packaged and deployed as a single unit.
Microservices	Each function runs as a small, independently deployable service.
SRP	Single Responsibility Principle: each unit has exactly one reason to change.
gRPC	A fast binary remote procedure call protocol for internal service calls.
API Gateway	The single entry point for clients; it routes to the right service.
Hallucination	When an LLM generates text that sounds plausible but is false.

Symbols at a glance.

Symbol	Meaning
$\mathbf{u}, \mathbf{v}$	Two embedding vectors (lists of numbers).
$\mathbf{u} \cdot \mathbf{v}$	Dot product: sum of element-wise products; measures alignment.
$\ \mathbf{u}\ $	Length (norm) of vector $\mathbf{u}$; $\ \mathbf{u}\ = \sqrt{u_1^2 + u_2^2 + \dots}$.
$\cos(\theta)$	Cosine similarity; 1 = same direction (very similar), 0 = unrelated.
$O(\log N)$	HNSW search time; grows slowly with number of stored vectors N .
